

BUILDING APPS WITH GEMINI

O'Reilly Live Training — Quick Reference Cheatsheet

google-genai SDK | Gemini 3.5 Flash | Python 3.10+ | 2026

1. SETUP & AUTHENTICATION

Install & Configure

```
# Install
pip install google-genai python-dotenv

# .env file
GEMINI_API_KEY=your_key_here
# OR: GOOGLE_API_KEY=your_key_here
```

Initialize Client

```
from google import genai
from google.genai import types
import os

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)
```

2. TEXT GENERATION

Basic Generation

```
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="Explain transformers in 3 bullets"
)
print(response.text)
```

With Config (Temp/Top-K/Top-P)

```
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="Write a haiku about Python",
    config=types.GenerateContentConfig(
        temperature=0.9,
        top_k=40,
        top_p=0.99,
        max_output_tokens=256,
    )
)
print(response.text)
```

3. MULTIMODAL: IMAGES & DOCUMENTS

Image — Upload API

```
# Upload once, reuse in multiple requests
my_file = client.files.upload(
    file="./photo.png"
)
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=[my_file, "Caption this image."]
)
print(response.text)
```

Image — Inline Bytes

```
with open("photo.png", "rb") as f:
    image_bytes = f.read()

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=[
        types.Part.from_bytes(
            data=image_bytes,
            mime_type="image/png",
        ),
        "What is in this image?"
    ]
)
print(response.text)
```

PDF — From URL

```
import httpx

doc_data = httpx.get(
    "https://arxiv.org/pdf/1706.03762"
).content

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=[
        types.Part.from_bytes(
            data=doc_data,
            mime_type="application/pdf",
        ),
        "Summarize in bullet points"
    ]
)
print(response.text)
```

PDF — Local File

```
import pathlib

filepath = pathlib.Path("./paper.pdf")

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=[
        types.Part.from_bytes(
            data=filepath.read_bytes(),
            mime_type="application/pdf",
        ),
        "Extract key insights"
    ]
)
print(response.text)
```

4. STRUCTURED OUTPUT (PYDANTIC + JSON SCHEMA)

Define Schema & Parse

```
from pydantic import BaseModel, Field

class Table(BaseModel):
    table_title: str = Field(
        description="title of the table"
    )
    table_contents: dict = Field(
        description="table data as a dict"
    )

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=[pdf_part, "Extract all tables"],
    config={
        "response_mime_type": "application/json",
        "response_schema": list[Table],
    }
)
tables = response.parsed # list[Table]
```

Extract Sections from Text

```
class Section(BaseModel):
    title: str
    content: str

def extract_sections(text: str):
    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=(
            "Extract main sections "
            f"from:\n\n{text}"
        ),
        config={
            "response_mime_type":
                "application/json",
            "response_schema": list[Section],
        },
    )
    return response.parsed

sections = extract_sections(report_text)
for s in sections:
    print(s.title)
```

5. EMBEDDINGS & SEMANTIC SEARCH

Generate Embeddings

```
# Single embedding
result = client.models.embed_content(
    model="gemini-embedding-2",
    contents="What is the meaning of life?"
)
vector = result.embeddings[0].values

# With task type (recommended)
result = client.models.embed_content(
    model="gemini-embedding-2",
    contents="search query text",
    config=types.EmbedContentConfig(
        task_type="RETRIEVAL_QUERY"
    )
)
```

Embedding Task Types

- **RETRIEVAL_DOCUMENT** — indexing docs into a store
- **RETRIEVAL_QUERY** — encoding a search query
- **SEMANTIC_SIMILARITY** — comparing text pairs
- **CLASSIFICATION** — text classification tasks
- **CLUSTERING** — grouping similar documents

6. SIMPLE RAG WITH EMBEDDINGS

Build Document Store & Semantic Search (dot product)

```
import numpy as np, pandas as pd

# 1. Build store – embed each document once
df = pd.DataFrame({"Title": titles, "Text": texts})
df["Embeddings"] = df.apply(
    lambda row: client.models.embed_content(
        model="gemini-embedding-2",
        contents=f"Title: {row['Title']}. Content:\n{row['Text']}",
        config=types.EmbedContentConfig(task_type="RETRIEVAL_DOCUMENT")
    ).embeddings[0].values,
    axis=1,
)

# 2. Search – cosine via dot product (embeddings are unit-normalised)
def find_best_passage(query: str, dataframe: pd.DataFrame) -> str:
    q_emb = client.models.embed_content(
        model="gemini-embedding-2",
        contents=query,
        config=types.EmbedContentConfig(task_type="RETRIEVAL_QUERY")
    ).embeddings[0].values
    scores = np.dot(np.stack(dataframe["Embeddings"]), q_emb)
    return dataframe.iloc[np.argmax(scores)]["Text"]

# 3. Answer with context
passage = find_best_passage("best prompting techniques?", df)
answer = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=f"Answer using this context:\n{passage}\n\nQuestion: best prompting techniques?"
)
print(answer.text)
```

7. TOKEN COUNTING & USAGE

Count Tokens (Before Generation)

```
total = client.models.count_tokens(
    model="gemini-3.5-flash",
    contents="The quick brown fox."
)
print(total.total_tokens) # e.g. 10
```

Usage Metadata (After Generation)

```
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="Explain AI in 50 words"
)
meta = response.usage_metadata
print(meta.prompt_token_count)
print(meta.candidates_token_count)
print(meta.total_token_count)
```

8. MODELS REFERENCE

Available Gemini Models

Model ID	Best For	Notes
<code>gemini-3.5-flash</code>	Current default — text & multimodal tasks	Default choice for dev & production
<code>gemini-3.1-pro-preview</code>	Pro tier — complex reasoning, PDF analysis	Highest capability, higher cost
<code>gemini-3.1-flash-image</code>	Image generation	Specialized for image synthesis tasks
<code>gemini-embedding-2</code>	Recommended — multimodal (text, images, video, audio, PDFs)	Use with <code>task_type</code> for best results
<code>gemini-embedding-001</code>	Stable — text-only (legacy)	Production-ready embedding model

9. PROMPT ENGINEERING TECHNIQUES

Core Techniques

- **Zero-Shot** — Direct task, no examples. Start here.
- **Few-Shot** — 3–5 input/output examples. Best for format control.
- **Role Prompting** — "Act as a senior Python developer..." Shapes tone & expertise.
- **System Prompt** — Sets global rules: "Only return JSON", "Be concise".
- **Contextual Prompt** — Inject relevant background before the question.

Advanced Techniques

- **Chain of Thought (CoT)** — Append "Let's think step by step." Use `temp=0`.
- **Self-Consistency** — Run CoT N times at higher temp, take majority answer.
- **Step-Back Prompting** — Ask the general question first, use answer as context.
- **ReAct** — Reason + Act: interleave Thought / Action / Observation loops for agents.
- **APE** — Use LLM to generate & evaluate prompt variants automatically.

10. MODEL CONFIGURATION PARAMETERS

Sampling Controls

Parameter	Range	Low Value	High Value	Recommended Starting Point
Temperature	0 – 1.0	Deterministic, factual	Creative, diverse	0.2 (factual) 0.9 (creative) 0 (CoT)
Top-K	1 – 40+	Conservative (20)	More diverse (40+)	30 as a balanced default
Top-P	0 – 1.0	Tighter logic (0.9)	More variety (0.99)	0.95 as balanced default
Max Output Tokens	1 – model max	Controls cost & latency. Too short truncates JSON — add buffer.		Set explicitly for production

11. PROMPT WRITING BEST PRACTICES

Do This

- Start with strong action verbs: *Summarize, Classify, Extract, Translate*
- Be specific about format: "Return a JSON array with fields: name, score"
- Use positive instructions — tell the model what TO do
- Add examples (few-shot) when format matters
- Set temp=0 for deterministic/reasoning tasks (math, CoT)
- Use variables: `{city}` , `{user_input}` for reusability
- Document your prompts: version, model, settings, output

Avoid This

- Vague instructions: "Write something about AI" → too broad
- Negative-only constraints: "Don't mention X" → weaker than "Focus only on Y"
- Assuming CoT output → always set temp=0 for reasoning
- Ignoring token limits → JSON gets truncated; use `json-repair` as safety net
- Using prompts without testing on edge cases
- Skipping the iterative loop: Craft → Test → Analyze → Document → Refine

12. KEY RESOURCES

Official Docs

- Quickstart: `ai.google.dev/gemini-api`
- Images: `.../image-understanding`
- PDFs: `.../document-processing`
- Embeddings: `.../embeddings`
- Audio: `.../audio`
- Video: `.../video-understanding`

Multimodal MIME Types

- `image/png` — PNG images
- `image/jpeg` — JPEG images
- `application/pdf` — PDF docs
- `audio/mp3` — Audio files
- `video/mp4` — Video files
- `text/plain` — Plain text

Useful Packages

- `google-generativeai` — Official SDK
- `httpx` — Fetch PDFs from URLs
- `pathlib` — Local file paths
- `pydantic` — Structured output
- `numpy` — Embedding math
- `pandas` — Document store
- `json-repair` — Fix truncated JSON

💡 API Key Tips

Both `GOOGLE_API_KEY` and `GEMINI_API_KEY` are accepted — if both are set, `GOOGLE_API_KEY` takes precedence. Get your key at aistudio.google.com/app/apikey. Use `python-dotenv` to load from `.env` files in development.